

3.6 Lab: Linear Regression



3.6.1 Importing Packages

Ở cấp đầu của notebook, ta import các library chuẩn thường dùng.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
```

Các import mới

Trong lab này, ta sẽ giới thiệu các function và library mới. Chúng được import ở đây để nhấn mạnh những code object mới của lab. Đặt import gần đầu notebook giúp code dễ đọc: chỉ cần nhìn vài dòng đầu là biết các library được dùng.

```
In [2]: import statsmodels.api as sm
```

Chi tiết liên quan của các function sẽ được trình bày khi cần.

Ngoài import toàn bộ module, ta có thể chỉ import một vài item để giữ *namespace* gọn gàng. Ta sẽ dùng một số object cụ thể từ package *statsmodels*.

namespace
statsmodels

```
In [3]: from statsmodels.stats.outliers_influence \
import variance_inflation_factor as VIF
from statsmodels.stats.anova import anova_lm
```

Vì một import statement khá dài, ta chèn line break `\` để dễ đọc.

Ta cũng dùng một số function được viết riêng cho lab của sách trong package *ISLP*.

```
In [4]: from ISLP import load_data
from ISLP.models import (ModelSpec as MS,
                        summarize,
                        poly)
```

Kiểm tra Object và Namespace

Function *dir()* cung cấp danh sách object trong một namespace.

dir()

```
In [5]: dir()
```

```
Out[5]: ['In',
         'MS',
         '_',
         '___',
         '___',
         '__builtin__',
         '__builtins__',
         ...,
         'poly',
         'quit',
         'sm',
         'summarize']
```

Kết quả cho thấy mọi thứ Python tìm thấy ở top level. Một số object như `__builtins__` chứa reference đến built-in function như `print()`.

Mỗi Python object có namespace riêng, cũng truy cập bằng `dir()`; nó gồm attribute và method của object. Chẳng hạn, danh sách của một array có `'sum'`.

```
In [6]: A = np.array([3,5,11])
        dir(A)
```

```
Out[6]: ...
        'strides',
        'sum',
        'swapaxes',
        ...
```

Điều này chỉ ra object `A.sum` tồn tại; đây là method tính tổng array, có thể kiểm tra bằng `A.sum?`.

```
In [7]: A.sum()
```

```
Out[7]: 19
```

3.6.2 Simple Linear Regression

Trong mục này, ta dựng model matrix (còn gọi là design matrix) bằng transform `ModelSpec()` từ `ISLP.models`.

Ta dùng Boston housing dataset trong package `ISLP`. Dataset ghi `medv` (median house value) cho 506 khu dân cư quanh Boston. Ta xây regression model dự đoán `medv` bằng 13 predictor như `rmvar` (số phòng trung bình mỗi nhà), `age` (tỷ lệ đơn vị do chủ sở hữu cư trú được xây trước năm 1940) và `lstat` (phần trăm hộ gia đình có socioeconomic status thấp). Công việc dùng `statsmodels`, một Python package triển khai nhiều regression method thông dụng. Package `ISLP` cung cấp function nạp dữ liệu đơn giản `load_data()`:

```
load_data()
```

```
In [8]: Boston = load_data("Boston")
Boston.columns
```

```
Out[8]: Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
              'rad', 'tax', 'ptratio', 'black', 'lstat', 'medv'],
              dtype='object')
```

Gõ `Boston`? để xem thêm thông tin về dữ liệu.

Đầu tiên, ta dùng `sm.OLS()` để chỉ định simple linear regression model với response `medv` và predictor duy nhất `lstat`. Model matrix có thể dựng thủ công.

```
In [9]: X = pd.DataFrame({'intercept': np.ones(Boston.shape[0]),
                          'lstat': Boston['lstat']})
X[:4]
```

```
Out[9]:   intercept  lstat
0         1.0    4.98
1         1.0    9.14
2         1.0    4.03
3         1.0    2.94
```

Sau đó, ta lấy response và fit mô hình.

```
In [10]: y = Boston['medv']
model = sm.OLS(y, X)
results = model.fit()
```

Lưu ý `sm.OLS()` không fit mô hình mà chỉ đặc tả nó; `model.fit()` mới thực hiện fitting.

Function `summarize()` của `ISLP` tạo bảng gọn gồm parameter estimate, standard error, *t*-statistic và *p*-value. Nó nhận một argument, chẳng hạn object `results` do method `fit()` trả về.

```
In [11]: summarize(results)
```

```
Out[11]:      coef  std err      t  P>|t|
intercept  34.5538    0.563  61.415    0.0
lstat      -0.9500    0.039 -24.528    0.0
```

Dùng Transformation: Fit và Transform

Mô hình trên chỉ có một predictor nên việc dựng `X` rất trực tiếp. Trong thực tế, ta thường fit nhiều predictor được chọn từ array hoặc data frame, có thể biến đổi biến trước khi fit, chỉ định interaction, hoặc mở rộng một biến thành một tập biến như polynomial. Package `sklearn` gọi loại tác vụ này là một *transform*. Transform object được tạo với một số parameter, rồi dùng hai method chính: `fit()` và `transform()`.

Library `ISLP` cung cấp transform `ModelSpec()` (đã đổi tên thành `MS()` ở phần đầu) để đặc tả mô hình và dựng model matrix. `MS()` tạo transform object; cập method `fit()` và `transform()` dựng model matrix tương ứng.

Với simple regression dùng `lstat`, ta tạo transform bằng `design = MS(['lstat'])`. Method `fit()` nhận array gốc và có thể thực hiện các phép tính ban đầu do transform quy định, chẳng hạn mean và standard deviation để centering, scaling. Method `transform()` áp dụng transformation đã fit và tạo model matrix.

```
In [12]: design = MS(['lstat'])
design = design.fit(Boston)
X = design.transform(Boston)
X[:4]
```

```
Out[12]:  intercept  lstat
0         1.0    4.98
1         1.0    9.14
2         1.0    4.03
3         1.0    2.94
```

Ở trường hợp đơn giản này, `fit()` chỉ kiểm tra biến `'lstat'` có trong `Boston`; `transform()` dựng matrix gồm intercept và `lstat`.

Hai thao tác có thể gộp bằng method `fit_transform()`.

```
In [13]: design = MS(['lstat'])
X = design.fit_transform(Boston)
X[:4]
```

```
Out[13]:  intercept  lstat
0         1.0    4.98
1         1.0    9.14
2         1.0    4.03
3         1.0    2.94
```

Giống khi thực hiện riêng hai bước, object `design` bị thay đổi bởi thao tác `fit()`. Sức mạnh của pipeline này sẽ rõ hơn với mô hình phức tạp có interaction và transformation.

Object `results` có nhiều method cho inference. `summarize()` trình bày phần cốt lõi; method `summary()` cho bản tổng hợp đầy đủ hơn (output không hiển thị).

```
In [14]: results.summary()
```

Fitted coefficient cũng có thể lấy từ attribute `params`.

```
In [15]: results.params
```

```
Out[15]: intercept    34.553841
lstat              -0.950049
dtype: float64
```

Method `get_prediction()` tạo prediction, confidence interval và prediction interval của `medv` tại các giá trị `lstat`. Trước tiên, ta tạo data frame mới chứa những giá trị cần dự đoán, rồi dùng `design.transform()` dựng model matrix tương ứng.

```
In [16]: new_df = pd.DataFrame({'lstat': [5, 10, 15]})
newX = design.transform(new_df)
newX
```

```
Out[16]:   intercept   lstat
0         1.0         5
1         1.0        10
2         1.0        15
```

Tiếp theo, ta tính prediction tại `newX` và lấy attribute `predicted_mean`.

```
In [17]: new_predictions = results.get_prediction(newX);
new_predictions.predicted_mean
```

```
Out[17]: array([29.80359411, 25.05334734, 20.30310057])
```

Confidence interval của predicted value được tạo như sau.

```
In [18]: new_predictions.conf_int(alpha=0.05)
```

```
Out[18]: array([[29.00741194, 30.59977628],
               [24.47413202, 25.63256267],
               [19.73158815, 20.87461299]])
```

Để tính prediction interval, đặt `obs=True`:

```
In [19]: new_predictions.conf_int(obs=True, alpha=0.05)
```

```
Out[19]: array([[17.56567478, 42.04151344],
               [12.82762635, 37.27906833],
               [ 8.0777421 , 32.52845905]])
```

Với `lstat=10`, confidence interval 95% là (24.47, 25.63) và prediction interval 95% là (12.82, 37.28). Cả hai cùng có tâm tại predicted value 25,05 của `medv`, nhưng prediction interval rộng hơn đáng kể như kỳ vọng.

Tiếp theo, ta vẽ `medv` và `lstat` bằng `DataFrame.plot.scatter()`, rồi thêm regression line.

Định nghĩa Function

Package `ISLP` có sẵn function thêm đường vào plot, nhưng đây là dịp để ta tự định nghĩa function đầu tiên bằng `def`.

def

```
In [20]: def abline(ax, b, m):
          "Add a line with slope m and intercept b to ax"
          xlim = ax.get_xlim()
          ylim = [m * xlim[0] + b, m * xlim[1] + b]
          ax.plot(xlim, ylim)
```

Syntax là `def funcname(...)`. Function nhận `ax`, `b`, `m`, trong đó `ax` là axis object của plot có sẵn, `b` là intercept và `m` là slope. Ta có thể truyền thêm plotting option vào `ax.plot` bằng optional argument:

```
In [21]: def abline(ax, b, m, *args, **kwargs):
          "Add a line with slope m and intercept b to ax"
          xlim = ax.get_xlim()
          ylim = [m * xlim[0] + b, m * xlim[1] + b]
          ax.plot(xlim, ylim, *args, **kwargs)
```

`*args` cho phép số lượng tùy ý non-named argument; `**kwargs` cho phép số lượng tùy ý named argument như `linewidth=3`. Function chuyển nguyên các argument đó tới `ax.plot`. Bạn đọc muốn tìm hiểu thêm có thể xem mục defining functions tại docs.python.org/tutorial.

Ta dùng function mới để thêm regression line vào plot `medv` theo `lstat`.

```
In [22]: ax = Boston.plot.scatter('lstat', 'medv')
          abline(ax,
                  results.params[0],
                  results.params[1],
                  'r--',
                  linewidth=3)
```

Lời gọi cuối cùng tới `ax.plot()` tương đương `ax.plot(xlim, ylim, 'r--', linewidth=3)`. Argument `'r--'` tạo đường đỏ đứt, còn width bằng 3. Quan hệ giữa `lstat` và `medv` có dấu hiệu phi tuyến; ta sẽ khảo sát sau trong lab. Function có sẵn `ax.axline()` cũng thêm được đường, nhưng biết tự viết function giúp tạo display biểu cảm hơn.

Tiếp theo, ta xem diagnostic plot đã bàn trong Mục 3.3.3. Fitted value và residual là attribute của `results`; các influence measure được tính bằng method `get_influence()`. Vì không dùng component `fig` đầu tiên từ `subplots()`, ta chỉ nhận giá trị thứ hai vào `ax`.

```
In [23]: ax = subplots(figsize=(8,8))[1]

          ax.scatter(results.fittedvalues, results.resid)
          ax.set_xlabel('Fitted value')
          ax.set_ylabel('Residual')
          ax.axhline(0, c='k', ls='--');
```

Ta thêm đường ngang tại 0 bằng `ax.axhline()`, đặt màu đen `c='k'` và `ax.axhline()` linestyle đứt `ls='--'`. Residual plot (không hiển thị) cho một số bằng chứng phi tuyến. Leverage statistic với số predictor bất kỳ được lấy qua attribute `hat_matrix_diag` của kết quả `get_influence()`.

```
In [24]: infl = results.get_influence()
          ax = subplots(figsize=(8,8))[1]
          ax.scatter(np.arange(X.shape[0]), infl.hat_matrix_diag)
          ax.set_xlabel('Index')
          ax.set_ylabel('Leverage')
          np.argmax(infl.hat_matrix_diag)
```

Out [24]: 374

Function `np.argmax()` tìm index của phần tử lớn nhất trong array, tùy chọn theo một axis. Ở đây ta tối đa trên toàn array để xác định observation có leverage statistic lớn nhất.

3.6.3 Multiple Linear Regression

Để fit multiple linear regression bằng least squares, ta lại dùng transform `ModelSpec()` dựng model matrix và response. Argument của `ModelSpec()` có thể rất tổng quát; ở đây danh sách column name là đủ. Trước tiên, ta fit với `lstat` và `age`.

```
In [25]: X = MS(['lstat', 'age']).fit_transform(Boston)
model1 = sm.OLS(y, X)
results1 = model1.fit()
summarize(results1)
```

```
Out[25]:
```

	coef	std err	t	P> t
intercept	33.2228	0.731	45.458	0.000
lstat	-1.0321	0.048	-21.416	0.000
age	0.0345	0.012	2.826	0.005

Dòng đầu được nén thành biểu thức ngắn gọn mô tả việc dựng `X`.

Dataset `Boston` có 12 predictor; gỡ tất cả để regression sẽ công kênh. Ta dùng shorthand `.columns.drop()`:

```
In [26]: terms = Boston.columns.drop('medv')
terms
```

```
Out[26]: Index(['crim', 'zn', 'indus', 'chas', 'nox', 'rm', 'age', 'dis',
               'rad', 'tax', 'ptratio', 'lstat'],
              dtype='object')
```

Giờ có thể fit mô hình với mọi biến trong `terms` bằng cùng model-matrix builder.

```
In [27]: X = MS(terms).fit_transform(Boston)
model = sm.OLS(y, X)
results = model.fit()
summarize(results)
```

```
Out[27]:
```

	coef	std err	t	P> t
intercept	41.6173	4.936	8.431	0.000
crim	-0.1214	0.033	-3.678	0.000
zn	0.0470	0.014	3.384	0.001
indus	0.0135	0.062	0.217	0.829
chas	2.8400	0.870	3.264	0.001
nox	-18.7580	3.851	-4.870	0.000
rm	3.6581	0.420	8.705	0.000
age	0.0036	0.013	0.271	0.787
dis	-1.4908	0.202	-7.394	0.000
rad	0.2894	0.067	4.325	0.000
tax	-0.0127	0.004	-3.337	0.001
ptratio	-0.9375	0.132	-7.091	0.000
lstat	-0.5520	0.051	-10.897	0.000

Nếu muốn regression với mọi biến trừ một biến, chẳng hạn bỏ `age` vì `p-value` cao, syntax sau thực hiện fit mà không có predictor đó (output không hiển thị).

```
In [28]: minus_age = Boston.columns.drop(['medv', 'age'])
Xma = MS(minus_age).fit_transform(Boston)
model1 = sm.OLS(y, Xma)
summarize(model1.fit())
```

3.6.4 Multivariate Goodness of Fit

Ta truy cập từng component của `results` bằng tên; `dir(results)` cho biết những gì có sẵn. Vì vậy, `results.rsquared` cho R^2 , còn `np.sqrt(results.scale)` cho RSE.

Variance inflation factor ở Mục 3.3.3 đôi khi hữu ích để đánh giá ảnh hưởng của collinearity trong model matrix. Ta sẽ tính VIF của multiple regression fit và đồng thời giới thiệu *list comprehension*.

list comprehension

List Comprehension

Ta thường cần biến đổi một chuỗi object cho tác vụ khác. Ở đây, ta tính VIF cho từng feature trong X và tạo data frame có index khớp column của X . List comprehension giúp thao tác này gọn hơn. Python còn hỗ trợ dictionary và generator comprehension, nhưng nằm ngoài phạm vi.

Ta dùng function `variance_inflation_factor()` để tính VIF cho từng biến trong model matrix.

```
In [29]: vals = [VIF(X, i)
                for i in range(1, X.shape[1])]
vif = pd.DataFrame({'vif':vals},
                  index=X.columns[1:])
vif
```

```
Out[29]:
```

	vif
crim	1.767
zn	2.298
indus	3.987
chas	1.071
nox	4.369
rm	1.913
age	3.088
dis	3.954
rad	7.445
tax	9.002
ptratio	1.797
lstat	2.871

Function `VIF()` nhận data frame hoặc array cùng column index. Ta gọi nó tại chỗ cho mọi column của X , bỏ column 0 là intercept vì không quan tâm. Trong trường hợp này, các VIF không quá đáng chú ý.

Object `vals` cũng có thể được dựng bằng for loop:

```
In [30]: vals = []
for i in range(1, X.values.shape[1]):
    vals.append(VIF(X.values, i))
```

List comprehension giúp thực hiện thao tác lặp như vậy trực tiếp hơn.

3.6.5 Interaction Terms

Đưa interaction term vào linear model bằng `ModelSpec()` rất dễ. Tuple `("lstat", "age")` yêu cầu model-matrix builder thêm interaction giữa `lstat` và `age`.

```
In [31]: X = MS(['lstat',
                'age',
                ('lstat', 'age')]).fit_transform(Boston)
model2 = sm.OLS(y, X)
summarize(model2.fit())
```

```
Out[31]:
```

	coef	std err	t	P> t
intercept	36.0885	1.470	24.553	0.000
lstat	-1.3921	0.167	-8.313	0.000
age	-0.0007	0.020	-0.036	0.971
lstat:age	0.0042	0.002	2.244	0.025

3.6.6 Non-linear Transformation của Predictor

Model-matrix builder có thể chứa term ngoài column name và interaction. Function `poly()` của `ISLP` yêu cầu thêm các column biểu diễn polynomial function của argument đầu tiên.

```
In [32]: X = MS([poly('lstat', degree=2), 'age']).fit_transform(Boston)
model3 = sm.OLS(y, X)
results3 = model3.fit()
summarize(results3)
```

```
Out[32]:
```

	coef	std err	t	P> t
intercept	17.7151	0.781	22.681	0.000
poly(lstat, degree=2)[0]	-179.2279	6.733	-26.620	0.000
poly(lstat, degree=2)[1]	72.9908	5.482	13.315	0.000
age	0.0703	0.011	6.471	0.000

p -value gần bằng 0 của quadratic term (hàng thứ ba) gợi ý rằng nó cải thiện mô hình.

Mặc định, `poly()` tạo basis matrix gồm các *orthogonal polynomial*, được thiết kế cho phép tính least squares ổn định.¹ Nếu thêm `raw=True`, basis matrix chỉ gồm `lstat` và `lstat**2`. Cả hai basis đều biểu diễn quadratic polynomial nên fitted value không đổi, chỉ polynomial coefficient thay đổi. Mặc định, column do `poly()` tạo không có intercept vì `MS()` tự thêm.

Ta dùng `anova_lm()` để định lượng thêm mức quadratic fit vượt linear fit.

```
In [33]: anova_lm(results1, results3)
```

```
Out[33]:
```

	df_resid	ssr	df_diff	ss_diff	F	Pr(>F)
0	503.0	19168.13	0.0	NaN	NaN	NaN
1	502.0	14165.61	1.0	5002.52	177.28	7.47e-35

¹Thực chất, `poly()` là wrapper của workhorse function độc lập `Poly()` dùng để dựng model matrix.

`results1` là linear submodel chứa `lstat`, `age`; `results3` là mô hình lớn hơn có quadratic term của `lstat`. `anova_lm()` thực hiện hypothesis test so sánh hai mô hình. Null hypothesis nói quadratic term không cần thiết; alternative nói mô hình lớn tốt hơn. Ở đây $F = 177.28$ và p -value gần 0. F -statistic là bình phương t -statistic của quadratic term vì hai nested model chỉ khác một degree of freedom. Đây là bằng chứng rất rõ ràng quadratic polynomial của `lstat` cải thiện linear model, phù hợp dấu hiệu phi tuyến đã thấy giữa `medv` và `lstat`.

`anova_lm()` có thể nhận nhiều hơn hai nested model và so sánh từng cặp liên tiếp. Điều đó cũng giải thích các NaN ở hàng đầu: không có mô hình trước để so sánh.

```
In [34]: ax = subplots(figsize=(8,8))[1]
ax.scatter(results3.fittedvalues, results3.resid)
ax.set_xlabel('Fitted value')
ax.set_ylabel('Residual')
ax.axhline(0, c='k', ls='--')
```

Khi thêm quadratic term, residual hầu như không còn pattern rõ. Muốn fit polynomial bậc ba hoặc cao hơn, chỉ cần đổi argument `degree` của `poly()`.

3.6.7 Qualitative Predictors

Ở đây ta dùng dữ liệu `Carseats` trong package `ISLP`, dự đoán `Sales` (doanh số ghế trẻ em trên ô tô) tại 400 địa điểm bằng một số predictor.

```
In [35]: Carseats = load_data('Carseats')
Carseats.columns
```

```
Out[35]: Index(['Sales', 'CompPrice', 'Income', 'Advertising',
               'Population', 'Price', 'ShelveLoc', 'Age', 'Education',
               'Urban', 'US'],
              dtype='object')
```

Dữ liệu có qualitative predictor như `ShelveLoc`, chỉ chất lượng vị trí trưng bày ghế trong cửa hàng, với ba giá trị `Bad`, `Medium`, `Good`. Với qualitative variable như vậy, `ModelSpec()` tự tạo dummy variable, thường gọi là *one-hot encoding*. Các column của chúng có tổng bằng 1; để tránh collinearity với intercept, column đầu bị bỏ. Ở đây `ShelveLoc[Bad]` bị bỏ vì `Bad` là level đầu. Ta fit multiple regression model có một số interaction term.

one-hot
encoding

```
In [36]: allvars = list(Carseats.columns.drop('Sales'))
y = Carseats['Sales']
final = allvars + [('Income', 'Advertising'),
                  ('Price', 'Age')]
X = MS(final).fit_transform(Carseats)
model = sm.OLS(y, X)
summarize(model.fit())
```

```
Out[36]:
```

	coef	std err	t	P> t
intercept	6.5756	1.009	6.519	0.000
CompPrice	0.0929	0.004	22.567	0.000
Income	0.0109	0.003	4.183	0.000
Advertising	0.0702	0.023	3.107	0.002
Population	0.0002	0.000	0.433	0.665
Price	-0.1008	0.007	-13.549	0.000
ShelveLoc[Good]	4.8487	0.153	31.724	0.000
ShelveLoc[Medium]	1.9533	0.126	15.531	0.000
Age	-0.0579	0.016	-3.633	0.000
Education	-0.0209	0.020	-1.063	0.288
Urban[Yes]	0.1402	0.112	1.247	0.213
US[Yes]	-0.1576	0.149	-1.058	0.291
Income:Advertising	0.0008	0.000	2.698	0.007
Price:Age	0.0001	0.000	0.801	0.424

Ở dòng đầu, `allvars` được chuyển thành list để ta có thể thêm interaction term hai dòng sau. Model-matrix builder tạo `ShelveLoc[Good]` bằng 1 khi vị trí trưng bày tốt và 0 nếu không; `ShelveLoc[Medium]` bằng 1 khi vị trí trưng bày trung bình và 0 nếu không. Vị trí xấu tương ứng cả hai dummy variable đều bằng 0. Coefficient dương của `ShelveLoc[Good]` cho thấy vị trí tốt associated với sales cao hơn vị trí xấu. `ShelveLoc[Medium]` có coefficient dương nhỏ hơn: vị trí trưng bày trung bình cho sales cao hơn vị trí xấu nhưng thấp hơn vị trí tốt.